

Functions

Functions

- Functions are bits of codes that you can reuse
- Functions have a special syntax

```
function functionName(parameters) {  
    code you want to run  
}
```

Function Declaration

```
function welcomeMsg(msg) {  
    alert(msg)  
}  
  
function welcomeMsg() {  
    alert("Welcome to JavaScript!")  
}
```

Function call

- Declaring a function doesn't actual do anything
- You need to **call** the function
- Calling a function changes the program flow


```
var x = "Hello"  
welcomeMsg(x)  
x = "Goodbye"  
welcomeMsg(x)
```

```
function welcomeMsg(msg) {  
    alert(msg)  
}
```

Parameters

- Sometimes functions need some information in order to perform its "function"
- The names of the parameters are not important, as long as you are consistent

Return values

- Some function return values
- These values can be used in assignment statements or conditional expressions

Return values

```
var firstName = welcomeMsg("Hi")
```

```
function welcomeMsg(msg) {  
    alert(msg) ;  
    var name = prompt("What is your name?")  
    return name  
}
```


Review

- Whenever possible, use built in functions
- When you need to write your own function, try not to be too specific
- Function parameters can have any name

© Colleen van Lent
University of Michigan
School of Information

Unless otherwise noted, this work is licensed under the
CC BY-NC 4.0 license.



Code Placement

Where To Place the Code

- Now that you are going to start to write your own functions, it is easier to separate code from content
- JavaScript code can be placed in the body, head, or in an external file

In the head

- When JavaScript functions are declared in the head section they are separated from the content
- Use the script tag
- Have access to all of the document information (ids, classes, etc.)

Code can be placed in the <head>

```
<head>
  <script>
    function message() {
      alert("This alert box was called with the online event");
    }
  </script>
</head>
<body>
  <h1>Functions</h1>
  <script>
    message()
  </script>
</body>
```

In an External File

- When JavaScript functions are in a separate file it is possible to reuse the code in multiple files
- Don't use the script tag

In an External File

```
<head>
  <script src="js/two-external.js"></script>
</head>
<body>
  <h1>Functions</h1>
  <script>
    message ()
  </script>
</body>
```

Debugging Your Code

- As your code becomes more complex, make sure that you are using your debugger
- The console is your friend!!

CodePen

- If you work on your code on an online editor (e.g. CodePen) the software lets you separate HTML, CSS, and JavaScript without any links

Review

- JavaScript can appear in the head or body of your code
- Code can also be placed in an external js file
- Personally, development done in head and moved to external after testing

© Colleen van Lent
University of Michigan
School of Information

Unless otherwise noted, this work is licensed under the
CC BY-NC 4.0 license.

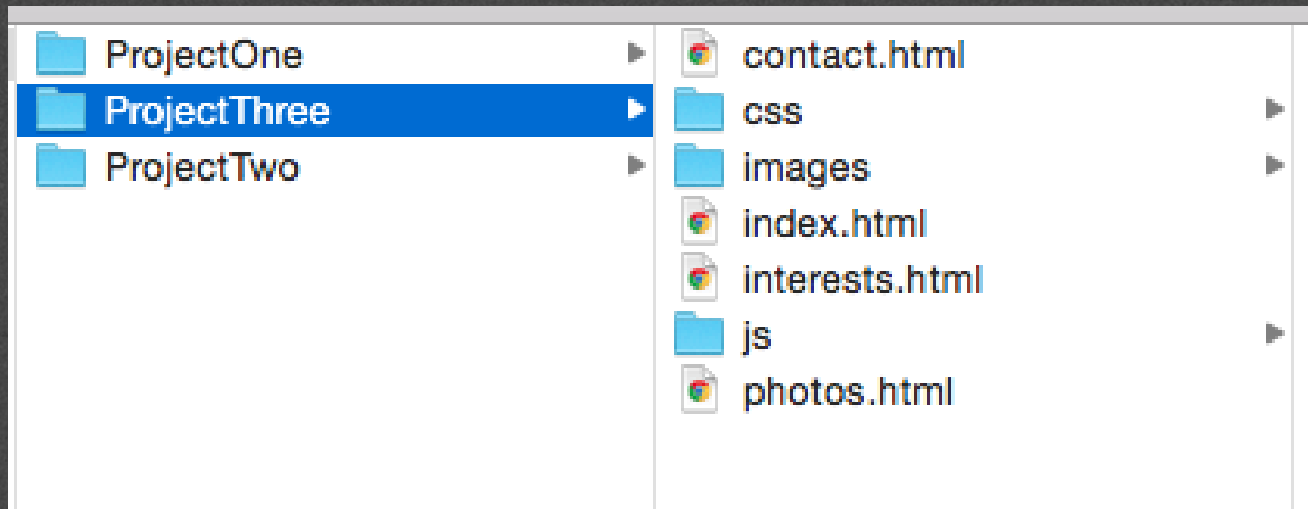
Folder Structure / Organizing Your Code

Folder Structure

- Web Developers tend to organize their code into separate parts
 - HTML
 - CSS
 - Images
 - JavaScript

Conventions

- Organizing your code is a convention, not a rule



Linking from an HTML file

```
<link rel = "stylesheet" href = "css/style.css">
```

```
<script src = "js/javaFunctions.js"></script>
```

```
<img src = "images/myPicture.jpg">
```


Linking from a CSS file

```
background: url("../images/holiday.png")
```

Debugging

- If a link isn't working you want to check a few things:
 - Did you spell the file names correctly? (Case matters!!)
 - Are files in the correct folder?
 - Are you working on the correct file?

© Colleen van Lent

University of Michigan

School of Information

Unless otherwise noted, this work is licensed under the
CC BY-NC 4.0 license.



Events

Adding the Interactivity!!

- It has been up to us to decide when the functions should execute
- It would be better if the functions were called based on special “events”
- The JavaScript API lets us add dynamic function calls!!

Events

- **onclick**
 - User clicks on an HTML element
- **onmouseover**
 - User moves the mouse over an HTML element
- **onresize**
 - browser window is resized
- **onload**
 - browser finishes loading the page

How it works

- Any element can react to an event.
- You need to add the event to the tag and include what you want to happen

```
<div onclick = "message()"> Clicking on this Div  
will invoke a JavaScript function</div>
```

Using Quotes

- You can use single quotes or double quotes for the event result
- Double quotes make it easier if you want to pass String parameters
- Be careful of copying and pasting quotes!

```
<div onclick = "message('Hi')">
```

Example

- Events – Basic Example
- Events – Basic Date Example

Events Change the Program Flow

- Some programs ran in a linear order (step-by-step)
- Events cause the program to “run continuously” since the DOM is always listening for events

More Events

- **Mouse Events**
 - onclick, ondblclick, onmousedown, onmouseenter, onmouseleave, onmousemove, onmouseout,....
- **Keyboard Events**
 - onkeydown, onkeypress, onkeyup
- **Frame Events**
 - onload, onresize, onscroll, onerror,...
- **Comprehensive list:**
 - <https://developer.mozilla.org/en-US/docs/Web/Events>

Review

- Without the events, JavaScript would be limited in ability to interact with the DOM
- Events are cool....they are also annoying
- Don't worry about memorizing the different events. As the need arises, look them up

© Colleen van Lent
University of Michigan
School of Information

Unless otherwise noted, this work is licensed under the
CC BY-NC 4.0 license.



Code With Me: Events

Coding in JavaScript Takes Practice

- Watching these videos isn't enough
- Dive into the code!
- Modify the code!
- Break the code!

Examples

- Events – Modify the DOM
- Events – Change Style

Review

Stop and Code!

© Colleen van Lent

University of Michigan

School of Information

Unless otherwise noted, this work is licensed under the
CC BY-NC 4.0 license.

“this”

Referring to Elements

- A key to smart programming is using functions
- A common roadblock is figuring out how to set up functions for reuse
 - How do I avoid writing a different function for every different element?
 - How can the function know which one I want to use?

“this”

- “this” is a keyword that allows an element to reference itself
 - Every object in the DOM has an automatically generated “this”
- Allows you to access an element’s info
 - Without “this” it would be difficult for the functions to know what data to use
- “this” is also used outside functions

Examples

- “this” Example – Simple
- “this” Example - Complex

Review

- “this” is a tricky concept to grasp
- Repeated practice helps
- If you get stuck, work backward from where you see the keyword find the last element that was started

© Colleen van Lent
University of Michigan
School of Information

Unless otherwise noted, this work is licensed under the
CC BY-NC 4.0 license.



Photo Gallery

Putting It into Practice

- Given the HTML code and the CSS code, can you:
 - change the background-image of an element?
 - change the content of an element?

Example

- Gallery Homework

background-image

- The background-image is an option for including graphics without using the img tag
- You should set a background-color as well in case the url isn't valid

```
background-image: url("mypic.jpg")  
background-color: #CCEECC;
```

Element text

- We have discussed two different ways to change the content
 - `document.write()`
 - `innerHTML()`

Tips

- The code should actually be quite short
- You will need to think about how to incorporate the quotes
- Remember, you use + to concatenate Strings

© Colleen van Lent
University of Michigan
School of Information

Unless otherwise noted, this work is licensed under the
CC BY-NC 4.0 license.